Bachelor Thesis

Titel

Plugin Architecture with Rust - an Analysis of Obstacles and Prospects

Problem

The Rust programming language is increasingly being selected for new projects where performance, reliability, and memory safety are critical. For many of these projects, such as data processing pipelines, application cores, or development tools, a plugin system is a fundamental requirement to ensure long-term extensibility. Rust's explicit lack of a stable Application Binary Interface (ABI) renders the conventional approach of dynamically linking libraries (*.so / *.dll), common in C/C++, unreliable and unsafe.

Research objective and questions

Goal of the paper is to analyze different approaches for developing a plugin system. The following points will be addressed:

- Illustrate the differences between compile time and dynamic time plugins with their respective advantages and disadvantages.
- How do the proposed options impact development time and complexity?
- Evaluate suitable tools and their inherent limitations, using specific examples such as WebAssembly (via interpreters), non-ABI-stable Rust formats, and others.

While not the primary focus, the thesis will also briefly address the performance implications of the discussed approaches.

Expected results and contributions

Upon completion of this bachelor thesis, the goal is to provide a clear and structured analysis of the current landscape for building plugin architectures in Rust. It will not present a single "best" solution, but rather a comprehensive comparison of available approaches. The thesis will serve as a guide for developers, enabling them to make an informed decision based on their specific requirements regarding safety, complexity, performance, and interoperability. The final document will delineate the obstacles posed by Rust's design and the prospects offered by modern solutions.

Using the result of the research, a *small protoype for data fusion track algorithmns* will be provided.